

Optimizing Talent Sourcing with AI: Distilling Large Language Models for Cost-Effective Recruitment and Job Matching

Abstract

This study explores the application of Large Language Models (LLMs) and model distillation to improve the precision and scalability of candidate-job matching processes. Using ChatGPT-4o for prompt-engineered extraction of detailed job requirements, this project distills the performance of a large model (ChatGPT-4o) into a cost-effective Llama-3.2-3B-Instruct model using QLoRA and Direct Preference Optimization (DPO). Using Jina V3 for semantic similarity measurements, we create a custom evaluation framework for measuring performance. Our experiments demonstrate that reformatted job descriptions and hyperparameter tuning yield significant performance improvements, achieving an F1 score of 0.788 for job description extraction against our benchmark of 0.9. While challenges remain our results highlight the potential for achieving similar levels of performance in a real world application at a significantly reduced cost.

Introduction

In the two-sided market of employment, the efficiency of filling vacant job positions with the best candidate not only affects the operating expenses on a firm's income statement, but also the lives of the countless job seekers searching for the perfect role. With recruitment costs driving companies to pay 15% - 30% commission¹ on salaries, the staffing industry was estimated at \$218 Billion in 2022². Despite this, the current recruitment sourcing process is guided by high level public job board profiles, while job seekers are left to sift through countless postings. This environment, driven by volume and surface-level matching criteria, fails to leverage the rich information that modern Natural Language Processing methods can provide.

Recent advancements in Large Language Models and semantic analysis present an opportunity to revolutionize these processes to match candidates in a more rigorous manner. This project aims to employ prompt-engineering techniques on the ChatGPT-4o model to extract detailed requirements from job descriptions. We then distill this capability into a smaller, cost-effective Llama 3.2 3B model using QLoRA, Direct Preference Optimization (DPO) and Boosting, enabling broader use with minimal performance degradation. Finally, we will measure the F1 scores utilizing Jina V3 embeddings to measure semantic similarity between extracted results. In doing so, this research seeks to improve efficiency in talent sourcing and enhance the precision of candidate-job matching criteria.

Background & Related Work

Major industry players have begun to explore the integration of advanced AI methodologies into their recruitment platforms, though current efforts remain limited and resource-bound. For instance, LinkedIn, widely recognized as a leading online professional network and job board, currently recommends job postings based on basic profile information, search history, and user activity³. While the platform has recently introduced premium AI-driven features, these enhancements currently operate on a single-job basis, rely on rudimentary profile data, and fail to

¹ Indeed. "Cost of Hiring Employees." October 31, 2024.

<https://www.indeed.com/hire/c/info/cost-of-hiring-employees?hl=en&co=us#3>.

² Statista. "Staffing Industry Market Size in the U.S. from 2012 to 2022." September 23, 2023.

<https://www.statista.com/statistics/873648/us-staffing-industry-market-size/>.

³ LinkedIn. "Jobs recommended for you" 2022. <https://www.linkedin.com/help/linkedin/answer/a512279>.

generate quantifiable metrics of fit. Similarly, Indeed has taken initial steps toward leveraging artificial intelligence by incorporating ChatGPT use on a limited scale⁴. These early-stage implementations highlight the industry's interest in leveraging AI capabilities into their systems, as practical constraints hinder their scale.

Li et al. introduced SkillGPT⁵, a cost-effective, open-source tool for skill extraction and standardization using the Vicuna-13B model. By combining text summarization and vector similarity search against precomputed ESCO (European Skills, Competences, Qualifications, and Occupations) embeddings, SkillGPT addresses challenges such as hallucinations and inefficiencies in handling lengthy job descriptions. This work highlights leveraging open-source LLMs for practical applications, aligning with this study's focus on cost-effective, scalable solutions through model distillation.

Herandi et al.'s Skill-LLM⁶ further advances skill extraction by fine-tuning general-purpose LLMs to improve accuracy and reliability. Their structured output format mitigates issues such as hallucinations and parsing errors common in zero-shot and few-shot LLM applications. Skill-LLM achieves superior F1 scores by leveraging LLMs' extensive pretraining while minimizing the need for domain-specific datasets. This approach underscores the synergy between fine-tuning and LLMs' inherent generalization capabilities, aligning with the broader effort to balance precision and scalability in AI-driven recruitment.

To address scalability constraints, artificial intelligence practitioners have turned to model distillation⁷, which allows for a large and more complex model to teach its capabilities to a more simple and lightweight model. By preserving the core learned representations in a more resource-efficient model, distillation strikes a balance between accuracy and scalability. The rise of Parameter Efficient Fine-tuning Techniques (PEFT) like QLoRA⁸ have added another dimension to this optimization effort by applying quantization and low-rank adaptation to further compress and efficiently fine-tune large models. This allows for most performance to be maintained while substantially lowering memory requirements and computational costs⁹. Similarly, developments in text embeddings allow for precise and measurable evaluation of semantic sentence similarity which can be utilized to assess matches. The combination of advanced large language models, model distillation, PEFT techniques, and robust semantic sentence similarity measures is setting the stage for a new era of scalable artificial intelligence in the staffing industry.

Approach

Data Overview

⁴ Indeed. "How Indeed Uses AI to Provide Better Matching Context for Job Seekers." August 15, 2024. <https://www.indeed.com/lead/how-indeed-uses-ai-to-provide-better-matching-context-for-job-seekers>.

⁵ Li, Nan, Bo Kang, and Tijl De Bie. "SkillGPT: A RESTful API Service for Skill Extraction and Standardization Using a Large Language Model." 2023. *arXiv*. <https://arxiv.org/abs/2304.11060>.

⁶ Herandi, Amirhossein, Yitao Li, Zhanlin Liu, Ximin Hu, and Xiao Cai. "Skill-LLM: Repurposing General-Purpose LLMs for Skill Extraction." 2024. *arXiv*. <https://arxiv.org/abs/2410.12052>.

⁷ Ping Yu, Jing Xu, Jason Weston, and Iliia Kulikov, "Distilling System 2 into System 1," *arXiv preprint*, 2024, <https://arxiv.org/abs/2407.06023>.

⁸ Dettmers, Tim, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. "QLoRA: Efficient Finetuning of Quantized LLMs." 2023. *arXiv*. <https://arxiv.org/abs/2305.14314>.

⁹ Tim Dettmer, Artidoro Pagnoni, Ari Holtzman, Luke Zettlemoyer, "QLoRA: Efficient Finetuning of Quantized LLMs," *arXiv preprint*, 2023, <https://arxiv.org/abs/2305.14314>

The dataset consists of a stratified random sample of 10,000 technology job postings from Dice.com's proprietary database, with 1,000 job descriptions randomly selected from each month between January 2024 and October 2024. The data is stored as unformatted text and due to resource constraints, job description lengths were limited to 4800 characters. This dataset was randomly split into two datasets totalling 5,000 examples each.

Label Creation through Prompt Engineering

To create our distillation labels, we evaluated the Llama 3.1 705B Instruct and ChatGPT-4o models, ultimately selecting ChatGPT-4o for its stronger general performance and structured output. We engineered a prompt to extract technical skills, education, credentials, and experience requirements, along with supporting details, and classify them as required or preferred. Technical skills included skill names and minimum required years, education included level and field of study, credentials included specific titles, and experience included a description, minimum years, and associated industry. Skills and experiences mentioned as responsibilities but not explicitly required were labeled as preferred. The prompt was also designed to account for complex relationships, such as implicit 'and/or' logic, related alternatives (e.g., "Bachelor's in Data Science or a related field"), and multiple job levels within one description. These categories were then organized into a structured JSON output. See *Appendix A* for full prompt.

We employed various prompt engineering strategies¹⁰ to refine instructions and achieve the desired extraction quality. A structured chain-of-thought approach decomposed complex tasks into manageable steps, while error analysis-based and few-shot prompting enhanced the model's understanding of intricate cases. Role-based and annotation guideline-based prompting defined the model's objectives and clarified key terms. Significant improvements were realized through decomposed prompting, which involved cleaning and reformatting textual descriptions prior to extraction (see *Appendix B* for reformatting prompt). We evaluated each iteration by manually reviewing 50 job descriptions for accuracy, adjusting prompts until the results met our expectations. Due to resource constraints and the absence of a labeled dataset we limited our sample size and did not employ formal evaluation metrics during this process. Additionally, although temperature was minimized, the lack of a fixed random seed introduced variability in the outputs, affecting the downstream performance of our fine-tuned models (*Appendix C*).

Model Fine Tuning, Experimentation and Evaluation

The following set of experiments and model trainings were conducted:

1. **Baseline Model:** The initial Llama-3B-Instruct model was evaluated without any fine-tuning, utilizing few-shot prompting. This served as the baseline for subsequent experiments.
2. **Grid Search Hyperparameter Tuning:** Hyperparameter tuning was performed using grid search on a fine-tuned QLoRA (Quantized Low-Rank Adaptation) model using a Rank of 16, Alpha of 32, and Rank-stabilized LoRA¹¹. The hyperparameters considered during this step included Learning Rates of [1e-5, 2e-5, 2e-4] and Dropout of [0, 0.1]. The following layers were finetuned: Query Projection, Key Projection, Value Projection, Output Projection, Gating Projection, Up Projection and Down Projection. These hyperparameters and overall QLoRA configuration were chosen based on the finding from

¹⁰ Shubham Vatsal and Harsh Dubey, "A Survey of Prompt Engineering Methods in Large Language Models for Different NLP Tasks," arXiv preprint, 2024, <https://arxiv.org/abs/2407.12994>.

¹¹ Kalajdzievski, Damjan. "A Rank Stabilization Scaling Factor for Fine-Tuning with LoRA." 2023. *arXiv*. <https://arxiv.org/abs/2312.03732>.

Dettmers, et al.¹². All other hyper parameters were set to the TRL default. The fine-tuning was conducted using the first 5,000 job descriptions with a test train split of 70%, 15% and 15% (3,500 Train, 750 Validation, 750 Test).

3. **Selection of Optimal Hyperparameters:** The best hyperparameters were identified by evaluating the f1 score on the validation dataset for each model configuration. The model configuration yielding the highest f1 score was selected for further training in the reformatted, expanding and direct preference optimization sections below.
4. **Fine-Tuning Reformatted Job Descriptions:** We also experimented with using the reformatted job description data, structured to help the model easily extract key information. This data was used as input to train the QLoRA model, with the output being JSON-formatted labels. The goal was to test whether formatting raw job descriptions would improve the accuracy of the QLoRA model.
5. **Training with Expanded Dataset:** Using the best-performing hyperparameters, the model was further trained on an additional 5,000 job descriptions with 3,500 added to the training set and 750 to the validation set. Additionally, job descriptions from the original training dataset that had a F1 score below 0.6 were boosted by appending the low scoring records to the training data. This allowed the model to focus more on low scoring records, doubling their weight compared to other records.
6. **Direct Preference Optimization (DPO):** Using the highest-performing QLoRA model, we processed 5,000 unseen job descriptions to generate labels. These were evaluated alongside the true labels to identify underperforming examples (Threshold: F1 score ≤ 0.6). This yielded 1,804 examples, split into 70% training (1,262) and 30% validation (542) sets. Using the original prompts, descriptions, true labels, and QLoRA labels, DPO optimization was applied over the same 7 QLoRA layers to preserve the model's structure. Three experiments with varying hyperparameters (learning rates of [5e-7, 5e-6] and Betas of [0.1, 0.03], over two epochs) were conducted. These hyperparameters of Learning Rate of 5e-7 and Beta of 0.1 were chosen as they have been found in previous research to be the most performant DPO hyperparameters⁸. All other hyperparameters used TRL defaults.
7. **Scoring Overview:** To evaluate our experiments, we developed a custom scoring methodology that employs a modified F1 score that integrates semantic similarity, core attribute importance, and conditional operators to evaluate model extraction performance. For full details see *Appendix D*.

Results

Our validation results showed that reformatted descriptions using a learning rate of 2e-5 produced the best results with an overall F1 score of 0.785. Somewhat surprisingly, doubling the dataset and training on reformatted data was not the highest performing model. We believe that this is due to the indeterminate labeling from ChatGPT, the effects of which were in turn amplified when doubling the dataset. See *Appendix C* for further details and an estimation on the effects of this indeterminate labeling on our results.

Our highest performing DPO experiment (3) had an F1 score of 0.768, a learning rate of 5e-7 and a Beta of 0.03. We saw a 0.130 difference in F1 score between Experiment's 3 (F1: 0.768) and 4 (F1: 0.755) suggesting that fitting more closely to the preference labels at the expense of generalizability improved our results (lower Beta). We saw a marked reduction in performance when increasing the DPO learning rate to 5e-6 as Experiment 12 had the second lowest F1 score of all the experiments conducted (F1: 0.641) beating only the baseline model. With the exception of this experiment all other DPO experiments outperformed our Boosted Experiment 8 (F1: 0.731),

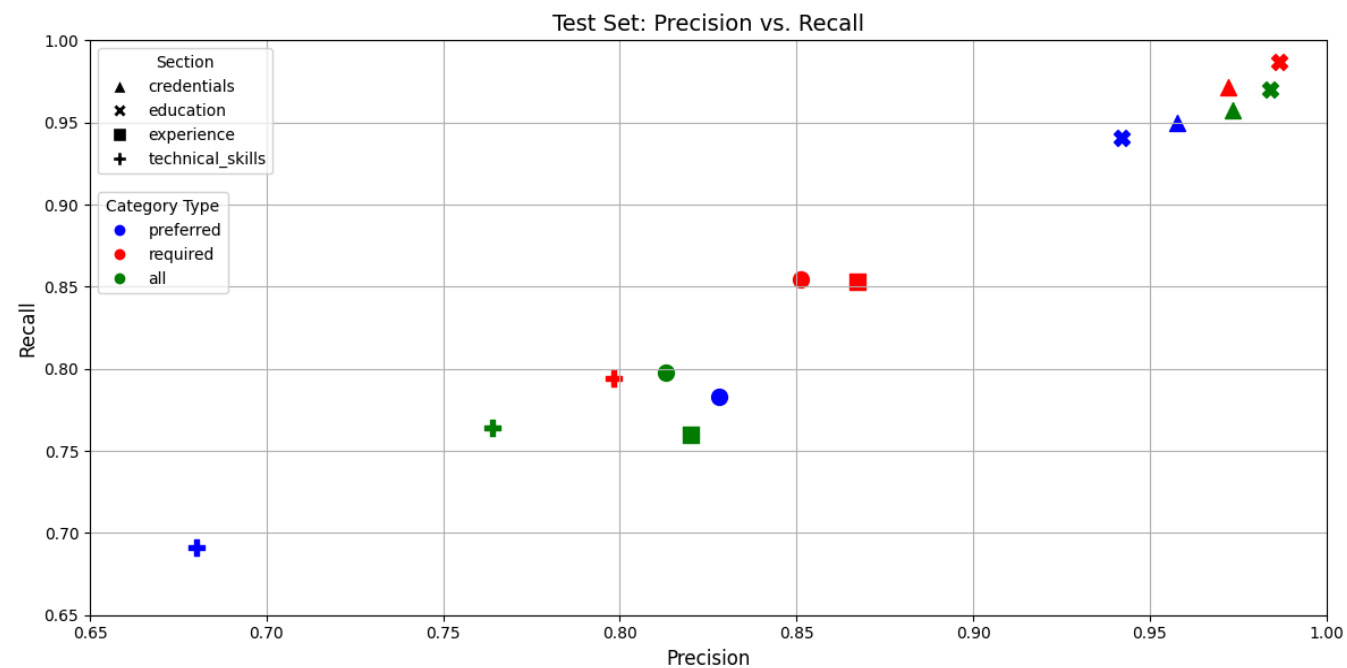
¹² Dettmers, Tim, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. "QLoRA: Efficient Finetuning of Quantized LLMs." 2023. *arXiv*. <https://arxiv.org/abs/2305.14314>.

suggesting an improvement gained specifically from the DPO process as compared to duplicating our low performing examples and repeating the QLoRA process.

Our best performing standard QLoRA experiment was Experiment 5 with an overall F1 score of 0.746 and whose Learning Rate and Drop Out hyperparameters were used in subsequent experiments. Overall, all of our models improved upon the baseline model’s performance, with our highest performing model demonstrating an F1 score increase of 0.516 and our lowest performing model demonstrating an F1 score increase of 0.374. See *Appendix E* for a table of full validation results details.

Using our best model hyperparameters of 2e-5 learning rate, dropout rate of 0.1, with reformatted descriptions, our final test f1 score was 0.788. See *Appendix F* for our full test results details.

Analysis



We observed that the model had the most difficulty with technical skills and experience and when analyzing the predictions with the lowest F1 scores, we saw that a key issue was that the model would extract the correct requirement but then misclassify it as either a technical skill or experience. This largely occurred when the word experience was used while describing a technical skill (e.g., "4 years of experience with Python") which confused the model on the correct category classification. We believe that the performance discrepancy between the preferred and required sections can be partially explained by our prompt which specified that any skills or experiences listed as responsibilities or not stated to be mandatory should be defaulted as preferred, due to the inexplicit nature of this rule. We find that education and credentials performed the best, as credentials were the most rare and education requirements were typically the shortest and most simple. We also found the model seemed to have the most trouble with more complex relationships including implicit and/or relationships or when “related” is proposed as an option to a requirement. When analyzing further, we found inconsistencies, duplicates, and errors in the labels which is a result of their indeterminate nature. To address these issues, we would need to refine the label prompt’s handling of complex relationships, create more clear language between

classification of technical skills & experience, and address the indeterminate nature of the labels by setting a random seed.

Future Work

Based on the results of this study, we identified areas for future improvement as follows:

1. **Simplified Prompts:** Simplifying prompt instructions to reduce ambiguity and improve extraction accuracy.
2. **Deterministic Outputs:** Setting a fixed random seed in the OpenAI API to minimize variability and improve consistency when creating labels from the teacher model.
3. **Domain-Specific Pre-training:** Pre-training the Llama model on job descriptions to improve understanding of job-related terminology and context.
4. **Larger Models:** Testing larger models like Llama 3.1 8B to evaluate improvements in handling complex requirements.
5. **Custom F1 Fine-Tuning:** Optimizing the model using the custom F1 score to enhance the goal of semantic similarity and precision.
6. **Proximal Policy Optimization (PPO):** Leveraging PPO for real-time preference adaptation, improving personalization and generalization.

Addressing these areas could refine data handling, boost model reliability, and advance scalable AI recruitment strategies.

Conclusion

This research demonstrates the potential of leveraging Large Language Models (LLMs) and model distillation to enhance the efficacy of extracting detailed requirement information from job descriptions. By distilling the capabilities of ChatGPT-4o into a smaller and cost-effective Llama-3.2-3B-Instruct model, using QLoRA and improving with Direct Preference Optimization (DPO), we achieved significant improvements in job requirement extraction accuracy, with our best-performing model achieving a test F1 score of 0.788. Our results show the capabilities of significant performance parity with large LLMs despite the limitations of using a smaller LLM specifically in the extraction of relevant information from unstructured job descriptions. In conclusion, this study establishes a foundation for further exploration of LLMs in developing talent matching strategies and provides a framework for creating more accurate and cost-efficient recruitment strategies that benefit both employers and job seekers.

Appendix

A. Labeling Prompt:

“You are a tool used to extract the requirements and responsibilities from a job description. I will be inputting a series of job descriptions, so please use the following prompt:

First, extract the technical skill requirements and responsibilities from the job description as JSON by following these steps:

Step 1 - Technical skills should be defined as: specific abilities that relate to the use of tools, technologies, programming languages, software applications, or technical processes.

Step 2 - Extract the technical skills that are explicitly stated in the job description. You must not include soft skills or job titles. You must include any technical skills that are listed as an experience requirement.

Step 3 - Store each of the technical skills in a key called 'skill_name' under an object called "technical_skills".

Step 4 - If multiple technical skills in a phrase are grouped using parentheses or listed using grouping connectors similar to "including", "like", "/", "or", or "such as" then list the skills in one array so that they result in a single skill_name. If technical skills are listed without grouping connectors and in a phrase ending in "and" then treat each listed skill as a separate skill_name. When skills are listed with no clear grouping implied, list each skill individually rather than in an array. Below are examples of desired outputs:

Example phrase 1 - "experience of 6+ years using cybersecurity architectures, IDS/IPS, SIEM tools, or firewalls"

Example output 1 - skill_name: [cybersecurity architectures, IDS, IPS, SIEM tools, firewalls]

Example phrase 2 - "knowledge of database systems including MySQL, PostgreSQL, and Redis"

Example output 2 - skill_name: [database systems, MySQL, PostgreSQL, Redis]

Example phrase 3 - "4+ years experience with Java/JEE, SOAP/REST/Micro Services, XML"

Example output 3 (3 separate skill_name keys) - skill_name: [Java, JEE], skill_name: [SOAP, REST, Micro Services], skill_name: [XML]

Example phrase 4 - "experience with SQL Server, Oracle, MongoDB"

Example output 4 (3 separate skill_name keys) - skill_name: [SQL Server], skill_name: [Oracle], skill_name: [MongoDB]

Example phrase 5 - "4 or more years experience with messaging architectures (EAI), SAP XI, BizTalk, and Web Services"

Example output 5 (4 separate skill_name keys) - skill_name: [messaging architectures, EAI], skill_name: [SAP XI], skill_name: [BizTalk], skill_name: [Web Services]

Example phrase 6 - "AWS Knowledge: Event Bridge, Cloud Watch, Cloud Trail"

Example output 6 (4 separate skill_name keys) - skill_name: [AWS], skill_name: [Event Bridge], skill_name: [Cloud Watch], skill_name: [Cloud Trail]

Example phrase 7 - "Working knowledge of EDB, Oracle, and AWS operations (S3, EC2)."

Example output 7 (3 separate skill_name keys) - skill_name: [EDB], skill_name: [Oracle], skill_name: [AWS Operations, S3, EC2]

Example phrase 8 - "expertise in the SAP Analytics Roadmap and new technologies such as SAP HANA, and SAP SAC."

Example output 8 (2 separate skill_name keys) - skill_name: [SAP Analytics Roadmap], skill_name: [SAP HANA, SAP SAC]

Example phrase 9 - "Demonstrated understanding of WAN, LAN, and VPN networks, including the relationship between network infrastructure and IP-enabled devices."

Example output 9 (4 separate skill_name keys) - skill_name: [WAN], skill_name: [LAN], skill_name: [VPN networks], skill_name: [network infrastructure, IP-enabled devices]

Step 5 - Classify each of the technical skills that are described as mandatory, essential, needed, or similar as 'required'. Classify the technical skills that are described as preferred, nice to have, or similar as 'preferred'. If a requirement is not mentioned or not clear, then classify it as 'preferred'. If a technical skill is only described under job responsibilities or job duties then classify it as 'preferred'. Separate the skills under two objects called 'required' and 'preferred'.

Step 6 - For each skill, extract the minimum years required explicitly stated for that specific skill (not inferred or through association) and list 0 if years are not explicitly stated for that specific skill. If a range of years is given, use the smallest year. Label these using a key called 'minyears'.

Step 7 - If any two technical skills you have extracted are extremely similar, merge them into a single skill_name array.

Next, extract the required level of education from the job description as JSON by following these steps:

Step 1 - Locate the requirements, qualifications, prerequisite, or a similar section of the job description and extract any educational requirements that are explicitly stated.

Step 2 - Create a key called 'education_level' that lists the levels of education mentioned in the job qualifications. Categorize the levels of education under one of the following values that fits best: High School Diploma, Associate's, Current Bachelor's Student, Bachelor's, Current Master's Student, Master's, Doctorate, Postdoctorate, Vocational, None.

Step 3 - Classify the levels of education that are explicitly described as mandatory, essential, required, or similar as 'required'. Classify the levels of education that are explicitly described as preferred, nice to have, or similar as 'preferred'. If a qualification is not mentioned, then classify the level of education as 'preferred'. Separate these levels of education under two objects called 'required' and 'preferred'.

Step 4 - If more than one level of education meets the necessary requirement, classify the higher levels of education as 'preferred'. If multiple levels of education are preferred, store them all as one level of education using an array of comma separated values. As a result, there should only be one 'education_level' key in each of the 'required' and 'preferred' object. For example, phrases like "A Bachelor's is required, a Master's/PhD is preferred" would have Bachelor's under the 'required' object but [Master's, Doctorate] under the 'preferred' object.

Step 5 - If experience is an acceptable alternative to an education requirement, add 'Or Experience' as a value into the 'education_level' array, but only if there is an existing education level. Below are examples of desired outputs:

Example phrase 1 - "a Master's degree in mathematics, STEM, or a related field, or 2 years of experience"

Example output 1 - education_level: [Master's, Or Experience]

Example phrase 2 - "graduate in information systems or equivalent hands-on work experience"

Example output 2 - education_level: [Bachelor's, Or Experience]

Example phrase 3 - "***Option A**": Bachelor's or higher degree with at least 30 semester hours in mathematics and physical sciences.- **Option B**": Combination of education and experience equivalent to a 4-year course of study."

Example output 3- education_level: [Current Bachelor's, Or Experience]

Step 6 - For each level of education in the list, identify and include any fields of study mentioned in a key called 'field_of_study' as separate values in an array. If terms like "or similar", "or equivalent", or "or related" are used, include 'Related' as a field of study. If no fields of study are listed then do not list anything. Below are examples of desired outputs:

Example phrase 1 - "A Bachelor's degree in Computer Science, STEM, or a related field"

Example output 1 - field_of_study: [Computer Science, STEM, Related]

Example phrase 2 - "A graduate in Computer Science, Business, STEM, or equivalent"

Example output 2 - field_of_study: [Computer Science, STEM, Related]

Next extract any required credentials, certifications, licenses, or similar credentials from the job description as JSON by following these steps:

Step 1 - Search within the job description for certifications, licenses, security clearances, or similar credential qualifications that are related to the job.

Step 2 - Extract the names of any certifications, licenses, security clearances, or similar credentials that are explicitly stated under keys called 'credential'. Do not include credential qualifications that are not related to the job like driver's licenses, citizenship documentation, or other unrelated credentials.

Step 3 - If multiple options are acceptable, place them in an array under the credential. If both a credential and it's abbreviation/acronym are given, provide both in the array. Below are examples of desired outputs:

Example phrase 1 - "Certified as IAT or IAM Level III"

Example output 1 - credential: [IAM Level III Certification, IAT Level III Certification]

Example phrase 2 - "CCNP (Cisco Certified Network Professional)"

Example output 2 - credential: [CCNP, Cisco Certified Network Professional]

Step 4 - Classify each of these credentials that are explicitly described as mandatory, essential, required, or similar as 'required'. Classify the credentials that are explicitly described as preferred, nice to have, or similar as 'preferred'. If a qualification is not mentioned, then classify the credential as 'preferred'. Separate these credentials under two objects called 'required' and 'preferred'.

Step 5 - If any certifications, licenses, security clearances, or similar credentials are listed as a technical skill, you must remove them from the technical skill section.

Step 6 - The output should be structured under the key 'Credentials', under objects for 'required' and 'preferred'.

Next, extract the experience requirements from the job description as JSON by following these steps:

Step 1 - Experience is defined as: The amount and type of prior work experience a candidate must have in order to qualify for the job or to complete the duties of the job. This includes the level of seniority, industry background, job responsibilities, and job duties but does not include technical skills.

Step 2 - Identify and extract any experience requirements that are explicitly stated in the job description.

Step 3 - Create a key for each experience requirement under a key called 'experience_desc'.

Step 4 - Review the experience requirements and if any of them involve technical skills then you must remove them.

Step 5 - If multiple experiences in a phrase are grouped using parentheses or listed using grouping connectors similar to "including", "like", "/", "or", or "such as" then you must list the experiences in one array so that they result in a single experience_desc. If experiences are listed without grouping connectors and listed in a phrase ending in "and" then you must treat each listed experience as a separate experience_desc. When no clear grouping is implied then you must list each experience individually rather than in an array. Below are examples of desired outputs:

Example phrase 1 - "7+ years overall experience in product management, marketing analysis, or professional services"

Example output 1 - experience_desc: [product management, marketing analysis, professional services]

Example phrase 2 - "must have leadership experience (manager, director, or VP level)"

Example output 2 - experience_desc: [manager, director, VP]

Example phrase 3 - "3+ years in designing, implementing, and optimizing custom solutions for enterprise software applications"

Example output 3 (3 separate experience_desc keys) - experience_desc: [designing custom solutions for enterprise software applications], experience_desc: [implementing custom solutions for enterprise software applications], experience_desc: [optimizing custom solutions for enterprise software applications]

Example phrase 4 - "1-2 years of experience leading software teams and working in an agile, kanban, or scrum environment"

Example output 4 (2 separate experience_desc keys) - experience_desc: [leading software teams],

experience_desc: [agile environment, kanban environment, scrum environment]

Example phrase 5 - "professional experience with version/source control, design/application, development principles/methodologies, integration tools"

Example output 5 (4 separate experience_desc keys) - experience_desc: [version control, source control],

experience_desc: [design, application], experience_desc: [development principles, development methodologies],

experience_desc: [integration tools]

Step 6 - If terms like "or similar", "or equivalent", or "or related" are used to describe an experience requirement you must include 'Related' in the array. Below are examples of desired outputs:

Example phrase 1 - "experience as a data scientist or a related role"

Example output 1 - experience_desc: [data scientist, Related]

Example phrase 2 - "5+ years of data engineering experience or equivalent"

Example output 2 - experience_desc: [data engineering, Related]

Step 7 - If experience is listed as a requirement without specifying a type of experience, then list the experience as 'Work Experience'. Below are examples of desired outputs:

Example phrase 1 - "4+ years of experience are required"

Example output 1 - experience_desc: [Work Experience]

Example phrase 2 - "Experience Required: 5 years"

Example output 2 - experience_desc: [Work Experience]

Step 8 - Identify the education requirements in the job description. For any phrasing that includes work experience as an alternative to an education requirement, such as "equivalent experience", "equivalent hands-on work experience", "relevant experience", or "professional experience" you must include an additional experience_desc key with the values [Work Experience, Or Education]. You must apply this rule to any phrasing where work experience is presented as an acceptable substitute for an education requirement. Below are examples of desired outputs:

Example phrase 1 - "An degree in engineering or 3+ years of relevant experience"

Example output 1 - experience_desc: [Work Experience, Or Education]

Example phrase 2 - "A Bachelor's degree in computer science (or equivalent experience) is needed"

Example output 2 - experience_desc: [Work Experience, Or Education]

Example phrase 3 - "Master of Science degree in computer science, MIS, or equivalent hands-on work experience"

Example output 3 - experience_desc: [Work Experience, Or Education]

Example phrase 4 - "A degree from an accredited university preferred, but related experience is also allowed"

Example output 4 - experience_desc: [Work Experience, Or Education]

Step 9 - For each experience_desc in the list, identify and include any industries mentioned in a key called 'industry' as separate values in an array. If terms like "or similar", "or equivalent", or "or related" are used, include 'Related' as an industry. If no industries are listed then do not list anything. Below are examples of desired outputs:

Example phrase 1 - "Minimum of 5+ years of experience as a Project Manager / Scrum Master in the AI/ML industry"

Example output 1 - industry: [AI Industry, ML Industry]

Example phrase 2 - "7 years of data analyst experience in SaaS"

Example output 2 - industry: [SaaS]

Example phrase 3 - "Proven leadership experience in fintech or a related industry"

Example output 3 - industry: [fintech, Related]

Step 10 - Classify each of the experiences that are explicitly described as mandatory, essential, required, or similar as 'required'. Classify the experiences that are explicitly described as preferred, nice to have, or similar as 'preferred'. If a requirement is not mentioned or not clear, then classify the experience as 'preferred'. If an experience is only described under job responsibilities or job duties then classify it as 'preferred'. Separate these experiences under two objects called 'required' and 'preferred'.

Step 11 - Review each experience, and if any experiences contain technical skills within them remove them from the experience requirements and then add them to the technical skills requirements.

Step 12 - If any industries under 'required' are described as preferred then remove the industry.

Example phrase 1 - "2+ years of experience in Information Technology, preferably in the semiconductor industry"

Example output 1 - industry: []

Example phrase 2 - "4 years of experience as a data engineer are required, within the finance industry preferred"

Example output 2 - industry: []

Step 13 - For each experience, list the minimum years of experience that is explicitly specified for that experience and list 0 if years are not explicitly specified. If a range of years is given, give the smallest year. Label these using a key called 'minyears'.

Lastly, clean and organize the extracted requirements and responsibilities by following these steps:

Step 1 - If the job description contains the requirements for multiple positions or multiple levels of a position, you must only keep the 'required' qualifications that are relevant to the lowest position or most entry level position and then treat the rest of the qualifications as 'preferred'.

Step 2 - Combine the JSON outputs from skills, education, credentials, and experience into one JSON.

Step 3 - Organize the data under two main objects: 'required' and 'preferred', moving the respective skills, education, credentials, and experience under them accordingly.

Step 4 - Output the combined JSON. “

B. Reformatting Prompt:

“You are a tool used to organize and reformat job descriptions into a structured markdown format that is optimized for information extraction from the ChatGPT 4o model. You must retain all information from the job description that is related to job requirements, job qualifications, job responsibilities, or job duties. You must exclude any information related to descriptions of the company, team, salary, travel, location, culture, application process, physical requirements/demands, or benefits. After, output the reformatted job description in markdown format.”

- C. **ChatGPT-4o Labeling:** Due to the indeterminate nature of our labels, we expected models to be unable to meet an F1 score of 1. To estimate the effect of our indeterminate labels on performance, we deterministically labeled a random sample of 500 descriptions using the same processing as the nondeterministically labeled data from the 1500 label test set. We then measured the difference between the deterministic and nondeterministic labeling using our match score pipeline. This analysis resulted in an F1 score of 0.9 with a 0.031 margin of error, given the 33% sample size and using a conservative population proportion estimate of 50%. Given this result we estimated the new benchmark for our test set F1 score performance at 0.9, although we acknowledge the distillation process may have amplified this difference in the student model.
- D. **Scoring Methodology:** The key performance indicator we used was a modified F1 score that accounts for semantic similarity, core attribute importance, and the conditional operators (e.g., and, or) in our labels. F1 was chosen as our primary evaluation metric as we believe both precision and recall are important factors in a real world extraction use case. To evaluate performance at a line item level, we developed a function that computes precision, recall, and F1 score between predictions and labels for each category and preference level, yielding a total of 15 distinct scores. Semantic similarity measurement was calculated using the Jina V3 text embeddings model, which at the time of writing is one of the top performing models on the semantic textual similarity benchmark¹³. Using the Jina embeddings we classified correct

¹³ <https://huggingface.co/spaces/mteb/leaderboard>

⁸ Hamish Ivison, , Yizhong Wang, Jiacheng Liu, Zeqiu Wu, Valentina Pyatkin, Nathan Lambert, Noah A. Smith, Yejin Choi, Hannaneh Hajishirzi. "Unpacking DPO and PPO: Disentangling Best Practices for Learning from Preference Feedback." (2024). <https://arxiv.org/html/2406.09279v2>

predictions as any meeting the semantic similarity threshold of .75 or above. To ensure the core attribute of each requirement (e.g., the skill name, education level, etc.) was weighted the same across categories, we set the weight of its classification as 50% for each requirement regardless of the number of supporting attributes (e.g., minimum years, industry, etc.).

E. Experiment Validation Set Results

Experiment Number	Learning Rate	Drop Out	Beta	Experiment	F1	Precision	Recall
0	2e-5	0.1	N/A	Reformatted	0.784564	0.806625	0.799916
1	2e-5	0	N/A	Reformatted	0.781106	0.804741	0.794177
2	2e-5	0	N/A	Double Reformatted Dataset	0.776238	0.787675	0.801865
3	5e-7	0	0.03	DPO	0.767630	0.801281	0.767166
4	5e-7	0	0.1	DPO	0.754590	0.787675	0.761062
5	2e-5	0.1	N/A	Standard	0.745785	0.763479	0.764517
6	2e-5	0	N/A	Standard	0.736745	0.731678	0.785958
7	1e-5	0.1	N/A	Standard	0.733678	0.727337	0.780885
8	2e-5	0	N/A	Boosted	0.731003	0.730503	0.774047
9	1e-5	0	N/A	Standard	0.729868	0.728891	0.773761
10	2e-4	0	N/A	Standard	0.729517	0.789340	0.713581
11	2e-4	0.1	N/A	Standard	0.727274	0.784232	0.713669
12	5e-6	0	0.1	DPO	0.641439	0.66163	0.669949
13	N/A	N/A	N/A	Baseline	0.267494	0.331057	0.263889

Results by top performing model

F. Test Set Results

Metric	Precision	Recall	F1 Score
Total	0.8130903384	0.7977880715	0.7883150172
Required	0.8512749791	0.854584751	0.8204081948
Preferred	0.8279822666	0.7829886993	0.7560662786
Technical Skills	0.7638981139	0.7641459818	0.7189163381
Education	0.9839405483	0.9698760234	0.9714275132

Credentials	0.9735134503	0.9573192546	0.9579172753
Experience	0.8201847218	0.7595681419	0.7313979659
Required Technical Skills	0.7981707638	0.794475035	0.7436317182
Required Education	0.9866666667	0.9866666667	0.9866666667
Required Credentials	0.9722666667	0.9713555556	0.9687
Required Experience	0.8671924385	0.8531199867	0.824977665
Preferred Technical Skills	0.6802096081	0.6909788588	0.6487218405
Preferred Education	0.9419047619	0.9404444444	0.9405777778
Preferred Credentials	0.9577025641	0.9497081855	0.9480711779
Preferred Experience	0.6613917554	0.6369660613	0.6147500255